

🤔 RP - 323 - Programmation fonctionnelle

[!TIP] > **Référence Javascript:** <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference>
Tester du code JS : <https://runjs.app/play>
Convertir en PDF : <https://marketplace.visualstudio.com/items?itemName=manuth.markdown-converter>

Table des matières

- Introduction
 - Objectifs du module / compétences
 - Enjeux dans mon métier
- Opérateurs javascript super-coool 😎
 - opérateur `?:`
 - opérateur `??`
 - opérateur `??=`
 - opérateur de décomposition 'spread' `...`
 - Déstructuration
- Date et Heure
 - Obtenir la date et/ou heure actuelle
- Math
 - `Math.PI` - la constante π
 - `Math.abs()` - la |valeur absolue| d'un nombre
 - `Math.pow()` - élever à une puissance
 - `Math.min()` - plus petite valeur
 - `Math.max()` - plus grande valeur
 - `Math.ceil()` - arrondir à la prochaine valeur entière la plus proche
 - `Math.floor()` - arrondir à la précédente valeur entière la plus proche
 - `Math.round()` - arrondir à la valeur entière la plus proche
 - `Math.trunc()` - supprime la virgule et retourne la partie entière d'un nombre
 - `Math.sqrt()` - la racine carrée d'un nombre
 - `Math.random()` - générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)
- JSON
 - `JSON.stringify()` - transformer un objet Javascript en JSON
 - `JSON.parse()` - transformer du JSON en objet Javascript
- Chaînes de caractères
 - `split()` - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau
 - `trim()`, `trimStart()` et `trimEnd()` - épuration des espaces en trop dans une chaîne (trimming)
 - `padStart()` et `padEnd()` - aligner le contenu dans une chaîne de caractères

- Console
 - `console.log()` - Afficher un message sur la console
 - `console.info()`, `warn()` et `error()` - Afficher un message sur la console (filtrables)
 - `console.table()` - Afficher tout un tableau ou un objet sur la console
 - `console.time()`, `timeLog()` et `timeEnd()` - Chronométrer une durée d'exécution
- Tableaux
 - `forEach` - parcourir les éléments d'un tableau
 - `entries()` - parcourir les couples index/valeurs d'un tableau
 - `in` - parcourir les clés d'un tableau
 - `of` - parcourir les valeurs d'un tableau
 - `find()` - premier élément qui satisfait une condition
 - `findIndex()` - premier index qui satisfait une condition
 - `indexOf()` et `lastIndexOf()` - premier/dernier élément qui correspond
 - `push()`, `pop()`, `shift()` et `unshift()` - ajouter/supprime au début/fin dans un tableau
 - `slice()` - ne conserver que certaines lignes d'un tableau
 - `splice()` - supprimer/insérer/remplacer des valeurs dans un tableau
 - `concat()` - joindre deux tableaux
 - `join()` - joindre des chaînes de caractères
 - `keys()` et `values()` - les clés/valeurs d'un objet
 - `includes()` - vérifier si une valeur est présente dans un tableau
 - `every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau
 - `fill()` - remplir un tableau avec des valeurs
 - `flat()` - aplatir un tableau
 - `sort()` - pour trier un tableau
 - `map()` - tableau avec les résultats d'une fonction
 - `filter()` - tableau avec les éléments passant un test
 - `groupBy()` - regroupe les éléments d'un tableau selon un règle
 - `flatMap()` - chaînage de `map()` et `flat()`
 - `reduce()` et `reduceRight()` - réduire un tableau à une seule valeur
 - `reverse()` - inverser l'ordre du tableau
- Techniques
 - ```(backticks) - pour des expressions intelligentes
 - `new Set()` - pour supprimer les doublons
- Fonctions
 - Déclaration de fonction
 - Fonctions immédiatement invoquées (IIFE)
- Conclusion

Introduction

Objectifs du module / compétences

- Comprendre les différences entre programmation impérative et fonctionnelle.
- Utiliser des fonctions pures, la composition de fonctions, l'immutabilité, et d'autres concepts clés du paradigme fonctionnel.
- Mettre en œuvre des fonctions fondamentales comme map, filter et reduce.
- Refactoriser du code impératif en code fonctionnel.
- Vérifier et améliorer la qualité et l'exactitude d'une implémentation fonctionnelle.
- Acquérir des bases solides en programmation fonctionnelle pour les appliquer dans des projets concrets.
- Réaliser une application Web selon les exigences d'un paradigme fonctionnel.
- Appliquer les bonnes pratiques et les patterns associés (comme le builder pattern, le currying ou les closures).
- Travailler en autonomie sur des exercices pratiques à l'aide de Visual Studio Code et GitHub.

Enjeux dans mon métier

La programmation fonctionnelle est une approche de plus en plus utilisée dans les projets modernes, notamment dans le traitement de données. Ce module est donc essentiel pour moi, car il me permet de penser et concevoir mon code autrement, de manière plus stricte, avec une meilleure lisibilité et maintenabilité. Apprendre à utiliser des fonctions pures, à éviter les effets de bord ou encore à manipuler les tableaux avec des fonctions comme map, filter ou reduce, me rend plus efficace. Ces compétences sont des atouts importants pour un développeur qui souhaite produire du code propre, testable et modulaire.

Opérateurs javascript super-coool 😎

opérateur ? :

L'expression `question?valeur1:valeur2` retournera `valeur1` si `question` vaut `true` sinon elle retournera `valeur2`.

```
const age = 15;  
const resultat = age >= 18 ? 'majeur' : 'mineur'; // 'mineur'
```

opérateur ??

Cet opérateur logique se nomme l'opérateur de "coalescence des nuls".

Renvoie son opérande de droite lorsque son opérande de gauche vaut `null` ou `undefined` et qui renvoie son opérande de gauche sinon.

```
const foo1 = null ?? 'default'; // "default"
const foo2 = 0 ?? 42; // 0
```

[!CAUTION] Contrairement à l'opérateur logique OU (`||`), l'opérande de gauche sera également renvoyé s'il s'agit d'une valeur équivalente à `false` et pas seulement `null` et `undefined`.

⚠ En d'autres termes **ATTENTION !!** lors de l'utilisation de `||` pour fournir une valeur par défaut à une variable, car on peut rencontrer des comportements inattendus lorsqu'on considère certaines valeurs comme correctes et utilisables (par exemple une chaîne vide `' '` ou `0`) !!

```
const foo3 = 0 || 42; // 42 => ATTENTION !
const foo4 = 1 || 42; // 1
const foo5 = null || 'salut !'; // 'salut !'
const foo6 = '' || 'salut !'; // 'salut !' => ATTENTION !
```

opérateur `??=`

Cet opérateur logique se nomme l'opérateur d'affectation de "coalescence des nuls", également connu sous le nom d'opérateur affectation logique nulle.

Évalue l'opérande de droite et l'attribue à gauche **UNIQUEMENT** si l'opérande de gauche est nulle (`null` ou `undefined`).

```
const a = { duration: 50 };
a.duration ??= 10; // pas fait
a.speed ??= 25; // fait => { duration: 50, speed: 25 }
```

opérateur de décomposition 'spread' `...`

L'opérateur de décomposition `spread` `...` permet de décomposer un itérable (comme un tableau) en en ses éléments distincts. Cela permet de rapidement copier tout ou une partie d'un tableau existant dans un autre tableau ou d'en extraire facilement des parties.

```
// Combiner des valeurs existantes dans un nouveau tableau
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
```

```
const numbersCombined = [...numbersOne, ...numbersTwo];

// Extraire uniquement ce qui est utile d'un tableau
const numbers = [1, 2, 3, 4, 5, 6];
const [one, two, ...rest] = numbers;

// Mariage d'objets avec mise à jour :-)
const myVehicle = {
  brand: 'Ford',
  model: 'Mustang',
  color: 'red',
};
const updateMyVehicle = {
  type: 'car',
  year: 2021,
  color: 'yellow',
};
const myUpdatedVehicle = { ...myVehicle, ...updateMyVehicle };
```

Déstructuration

L'opérateur de décomposition spread `...` sert aussi à isoler certains éléments afin de les utiliser ensuite, et de **mettre le reste** d'un coup ailleurs.

```
const valeurs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const [a, b, ...c] = valeurs;
console.log(a); // 1
console.log(b); // 2
console.log(c); // [3, 4, 5, 6, 7, 8, 9, 10]
```

Date et Heure

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Date

Obtenir la date et/ou heure actuelle

```
const maintenant = new Date(); // Obtenir l'un comme l'autre

console.log(maintenant.toLocaleDateString()); // ex: "06.06.2025"
console.log(maintenant.toLocaleTimeString()); // ex: "15:23:42"

const jour = maintenant.getDate();
const mois = maintenant.getMonth() + 1; // Attention : janvier = 0
const annee = maintenant.getFullYear();
const heure = maintenant.getHours();
const minute = maintenant.getMinutes();
const seconde = maintenant.getSeconds();
console.log(`${jour}/${mois}/${annee} - ${heure}h${minute}'`);

// Au format ISO (standard international)
console.log(maintenant.toISOString()); // ex: "2025-06-06T13:23:42.123Z"
```

Math

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math

Math.PI - la constante π

La constante Math.PI permet d'accéder directement à la valeur de π (pi), utile pour les calculs de cercles ou de trigonométrie.

```
console.log(Math.PI); // 3.141592653589793
```

Math.abs() - la |valeur absolue| d'un nombre

Retourne la valeur absolue (positive) d'un nombre, en supprimant le signe négatif s'il existe.

```
console.log(Math.abs(-10)); // 10
```

Math.pow() - élever à une puissance

Permet d'élever un nombre à une puissance donnée : `Math.pow(base, exposant)`.

```
console.log(Math.pow(2, 3)); // 8
```

`Math.min()` - plus petite valeur

Retourne la plus petite valeur parmi les arguments fournis.

```
console.log(Math.min(3, 7, -2, 0)); // -2
```

`Math.max()` - plus grande valeur

Retourne la plus grande valeur parmi les arguments fournis.

```
console.log(Math.max(3, 7, -2, 0)); // 7
```

`Math.ceil()` - arrondir à la prochaine valeur entière la plus proche

Arrondit un nombre à l'entier supérieur le plus proche.

```
console.log(Math.ceil(4.2)); // 5
```

`Math.floor()` - arrondir à la précédente valeur entière la plus proche

Arrondit un nombre à l'entier inférieur le plus proche.

```
console.log(Math.floor(4.9)); // 4
```

`Math.round()` - arrondir à la valeur entière la plus proche

Arrondit un nombre à l'entier le plus proche (selon sa virgule).

```
console.log(Math.round(4.4)); // 4  
console.log(Math.round(4.6)); // 5
```

`Math.trunc()` - supprime la virgule et retourne la partie entière d'un nombre

Supprime la partie décimale et retourne uniquement la partie entière.

```
console.log(Math.trunc(4.9)); // 4
console.log(Math.trunc(-4.9)); // -4
```

`Math.sqrt()` - la racine carrée d'un nombre

Retourne la racine carrée d'un nombre positif.

```
console.log(Math.sqrt(9)); // 3
```

`Math.random()` - générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)

Génère un nombre pseudo-aléatoire compris entre 0 (inclus) et 1 (exclus).

```
console.log(Math.random()); // ex: 0.721948321
```

JSON

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/JSON

`JSON.stringify()` - transformer un objet Javascript en JSON

Transforme un objet JavaScript en une chaîne de texte au format JSON. Très utile pour stocker ou transmettre des données.

```
const user = { nom: 'Alice', age: 25 };
console.log(JSON.stringify(user)); // '{"nom":"Alice","age":25}'
```

JSON.parse() - transformer du JSON en objet Javascript

Transforme une chaîne JSON valide en un objet JavaScript manipulable.

```
const json = '{"nom":"Alice","age":25}';  
console.log(JSON.parse(json)); // { nom: 'Alice', age: 25 }
```

Chaînes de caractères

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String

split() - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau

Divise une chaîne de caractères selon un séparateur donné et retourne un tableau.

```
const phrase = 'un-deux-trois';  
console.log(phrase.split('-')); // ['un', 'deux', 'trois']
```

trim(), trimStart() et trimEnd() - épuration des espaces en trop dans une chaîne (trimming)

Suppriment les espaces en début et/ou fin d'une chaîne.

```
const text = '  Hello World!  ';  
console.log(text.trim()); // 'Hello World!'  
console.log(text.trimStart()); // 'Hello World!  '  
console.log(text.trimEnd()); // '  Hello World!'
```

padStart() et padEnd() - aligner le contenu dans une chaîne de caractères

Permettent d'ajouter des caractères au début (padStart) ou à la fin (padEnd) d'une chaîne, jusqu'à atteindre une longueur donnée.

```
const num = '5';
console.log(num.padStart(3, '0')); // '005'
console.log(num.padEnd(4, '-')); // '5---'

const mot = 'Chat';
console.log(mot.padStart(8, ' ')); // '      Chat' ← aligné à droite
console.log(mot.padEnd(8, ' ')); // 'Chat      ' ← aligné à gauche
```

Console

Lien vers la documentation officielle : <https://developer.mozilla.org/fr/docs/Web/API/console>

`console.log()` - Afficher un message sur la console

```
console.log('Coucou !'); // Coucou !
```

`console.info()`, `warn()` et `error()` - Afficher un message sur la console (filtrables)

Ces méthodes affichent des messages avec un niveau d'importance différent. Utile pour distinguer les informations, avertissements et erreurs dans la console.

```
console.info('Chargement terminé.');// ⓘ Info (bleu ou neutre)
console.warn('Mémoire presque pleine.');// ⚠ Avertissement (jaune)
console.error('Fichier introuvable !');// ❌ Erreur (rouge)
```

`console.table()` - Afficher tout un tableau ou un objet sur la console

Affiche un tableau ou un objet sous forme de table lisible dans la console.

```
const personnes = [
  { nom: 'Alice', age: 25 },
  { nom: 'Bob', age: 30 },
  { nom: 'Charlie', age: 28 },
];
```

```
];  
  
console.table(personnes);  
  
console.table(personnes, ['nom']); // n'affiche que la colonne "nom"
```

console.time(), timeLog() et timeEnd() - Chronométrer une durée d'exécution

Permettent de mesurer le temps d'exécution d'un bloc de code entre le début (time()) et la fin (timeEnd()). timeLog() affiche le temps écoulé sans arrêter le chronomètre.

```
console.time('test'); // ⌚ Démarre le chrono  
for (let i = 0; i < 1e6; i++) {}  
console.timeLog('test'); // ⌚ Affiche le temps intermédiaire  
console.timeEnd('test'); // ⌚ Affiche le temps total et arrête le chrono
```

Tableaux

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array

forEach - parcourir les éléments d'un tableau

Permet d'exécuter une fonction pour chaque élément d'un tableau. Idéal pour parcourir un tableau sans créer de nouveau tableau.

```
const fruits = ['pomme', 'banane', 'cerise'];  
  
fruits.forEach((fruit, index) => {  
    console.log(`${index} : ${fruit}`);  
});
```

entries() - parcourir les couples index/valeurs d'un tableau

La méthode `.entries()` renvoie un itérateur contenant les paires `[index, valeur]` d'un tableau. C'est pratique pour parcourir à la fois la position et le contenu de chaque élément.

```
const fruits = ['pomme', 'banane', 'cerise'];

for (const [index, valeur] of fruits.entries()) {
  console.log(index, valeur);
}
// 0 'pomme'
// 1 'banane'
// 2 'cerise'
```

in - parcourir les clés d'un tableau

L'opérateur `for...in` permet de parcourir les index d'un tableau ou les propriétés d'un objet. Il est utile lorsqu'on veut connaître la position ou le nom des propriétés, plutôt que leur valeur.

```
const fruits = ['pomme', 'banane', 'cerise'];

for (const i in fruits) {
  console.log(i); // 0, 1, 2
}
```

of - parcourir les valeurs d'un tableau

La boucle `for...of` permet de parcourir directement les valeurs d'un tableau.

```
const fruits = ['pomme', 'banane', 'cerise'];

for (const fruit of fruits) {
  console.log(fruit);
}
// 'pomme'
// 'banane'
// 'cerise'
```

find() - premier élément qui satisfait une condition

Cette méthode retourne le premier élément d'un tableau qui vérifie une condition donnée. Si aucun élément ne correspond, elle renvoie `undefined`.

```
const nombres = [3, 7, 12, 18];
const premierPair = nombres.find((n) => n % 2 === 0);
console.log(premierPair); // 12
```

findIndex() - premier index qui satisfait une condition

La méthode `.findIndex()` renvoie l'index du premier élément d'un tableau qui respecte une condition. Si aucun élément ne correspond, elle renvoie -1.

```
const nombres = [5, 8, 12, 20];
const indexPair = nombres.findIndex((n) => n % 2 === 0);
console.log(indexPair); // 1 (car 8 est à la position 1)
```

indexOf() et lastIndexOf() - premier/dernier élément qui correspond

Ces méthodes renvoient la position (index) d'un élément dans un tableau.

`indexOf()` → cherche la première occurrence.

`lastIndexOf()` → cherche la dernière occurrence. Si l'élément n'existe pas, elles renvoient -1.

```
const fruits = ['pomme', 'banane', 'cerise', 'banane'];

console.log(fruits.indexOf('banane')); // 1
console.log(fruits.lastIndexOf('banane')); // 3
```

push(), pop(), shift() et unshift() - ajouter/supprime au début/fin dans un tableau

Ces méthodes servent à ajouter ou retirer des éléments d'un tableau :

`push()` → ajoute à la fin

`pop()` → supprime le dernier

`unshift()` → ajoute au début

`shift()` → supprime le premier

```
const fruits = ['pomme', 'banane'];
```

```
fruits.push('cerise'); // ['pomme', 'banane', 'cerise']
fruits.pop(); // ['pomme', 'banane']
fruits.unshift('kiwi'); // ['kiwi', 'pomme', 'banane']
fruits.shift(); // ['pomme', 'banane']
```

slice() - ne conserver que certaines lignes d'un tableau

La méthode `.slice()` permet de copier une partie d'un tableau sans modifier l'original. Elle prend deux index : le début inclus et la fin exclue.

```
const nombres = [10, 20, 30, 40, 50];
const partie = nombres.slice(1, 4); // [20, 30, 40]
```

splice() - supprimer/insérer/remplacer des valeurs dans un tableau

La méthode `.splice()` permet de modifier directement un tableau en supprimant, ajoutant ou remplaçant des éléments. Sa syntaxe : `.splice(indexDépart, nbASupprimer, ...élémentsAAjouter)`

```
const fruits = ['pomme', 'banane', 'cerise'];

// Supprimer 1 élément à partir de l'index 1
fruits.splice(1, 1); // ['pomme', 'cerise']

// Insérer un élément à l'index 1
fruits.splice(1, 0, 'kiwi'); // ['pomme', 'kiwi', 'cerise']

// Remplacer à l'index 2
fruits.splice(2, 1, 'mangue'); // ['pomme', 'kiwi', 'mangue']
```

concat() - joindre deux tableaux

La méthode `.concat()` permet de fusionner plusieurs tableaux en un seul sans modifier les originaux. Elle retourne un nouveau tableau contenant tous les éléments.

```
const a = [1, 2];
const b = [3, 4];
const c = a.concat(b); // [1, 2, 3, 4]
```

join() - joindre des chaînes de caractères

La méthode `.join()` transforme un tableau en une seule chaîne de texte, en insérant un séparateur entre chaque élément. Par défaut, le séparateur est une virgule `,`.

```
const mots = ['Bonjour', 'le', 'monde'];  
const phrase = mots.join(' '); // "Bonjour le monde"
```

`keys()` et `values()` - les clés/valeurs d'un objet

Ces méthodes permettent d'extraire les clés (keys) ou les valeurs (values) d'un objet sous forme de tableau. Elles sont très pratiques pour parcourir ou transformer un objet.

```
const personne = { nom: 'Alex', age: 25, ville: 'Fribourg' };  
  
console.log(Object.keys(personne)); // ['nom', 'age', 'ville']  
console.log(Object.values(personne)); // ['Alex', 25, 'Fribourg']
```

`includes()` - vérifier si une valeur est présente dans un tableau

La méthode `.includes()` vérifie si une valeur donnée existe dans un tableau. Elle renvoie `true` si l'élément est trouvé, sinon `false`.

```
const fruits = ['pomme', 'banane', 'cerise'];  
  
console.log(fruits.includes('banane')); // true  
console.log(fruits.includes('kiwi')); // false
```

`every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau

Ces méthodes testent si les éléments d'un tableau respectent une condition :

`.every()` → tous les éléments doivent passer le test → renvoie `true` ou `false`

`.some()` → au moins un élément doit passer le test → renvoie `true` ou `false`

```
const notes = [12, 15, 18, 10];  
  
console.log(notes.every((n) => n >= 10)); // true (toutes ≥ 10)  
console.log(notes.some((n) => n > 17)); // true (au moins une > 17)
```

fill() - remplir un tableau avec des valeurs

La méthode `.fill()` remplit tout ou une partie d'un tableau avec une valeur donnée. Elle modifie le tableau d'origine et peut prendre une plage d'index optionnelle.

```
const tableau = [1, 2, 3, 4, 5];  
tableau.fill(0, 1, 4); // [1, 0, 0, 0, 5]
```

flat() - aplatir un tableau

La méthode `.flat()` permet de fusionner les sous-tableaux dans un tableau principal. Par défaut, elle n'aplatit qu'un niveau, mais on peut préciser la profondeur.

```
const tableau = [1, [2, 3], [4, [5, 6]]];  
  
console.log(tableau.flat()); // [1, 2, 3, 4, [5, 6]]  
console.log(tableau.flat(2)); // [1, 2, 3, 4, 5, 6]
```

sort() - pour trier un tableau

La méthode `.sort()` trie les éléments d'un tableau en place (elle modifie l'original). Par défaut, elle trie comme des chaînes de caractères. Pour un tri numérique, il faut fournir une fonction de comparaison.

```
const nombres = [10, 2, 30];  
nombres.sort(); // [10, 2, 30] ❌ (tri alphabétique)  
nombres.sort((a, b) => a - b); // [2, 10, 30] ✅ (tri numérique)
```

map() - tableau avec les résultats d'une fonction

La méthode `.map()` crée un nouveau tableau en appliquant une fonction à chaque élément du tableau original. Elle ne modifie pas le tableau d'origine.

```
const nombres = [1, 2, 3, 4];  
const doubles = nombres.map((n) => n * 2); // [2, 4, 6, 8]
```

filter() - tableau avec les éléments passant un test

La méthode `.filter()` crée un nouveau tableau contenant uniquement les éléments qui passent un test (fonction retournant `true`). Elle ne modifie pas le tableau original.

```
const nombres = [5, 12, 8, 20];
const grands = nombres.filter((n) => n > 10); // [12, 20]
```

groupBy() - regroupe les éléments d'un tableau selon un règle

La méthode `.groupBy()` (ou `Object.groupBy()` en JS moderne) permet de classer les éléments d'un tableau dans des groupes selon une règle de tri définie par une fonction. Elle renvoie un objet dont les clés sont les catégories créées.

```
const nombres = [1, 2, 3, 4, 5, 6];
const groupes = Object.groupBy(nombres, (n) => (n % 2 === 0 ? 'pair' :
'impair'));

/*
{
  pair: [2, 4, 6],
  impair: [1, 3, 5]
}
*/
```

flatMap() - chaînage de map() et flat()

La méthode `.flatMap()` applique une transformation (`map`) à chaque élément, puis aplatit (`flat`) le résultat d'un niveau. C'est donc un raccourci de `.map(...).flat()`.

```
const phrases = ['Salut toi', 'Bonjour le monde'];
const mots = phrases.flatMap((p) => p.split(' '));
// ["Salut", "toi", "Bonjour", "le", "monde"]
```

reduce() et reduceRight() - réduire un tableau à une seule valeur

La méthode `.reduce()` sert à accumuler les valeurs d'un tableau pour obtenir un seul résultat (somme, objet, moyenne, etc.). `.reduceRight()` fait la même chose, mais de droite à gauche.

```
const nombres = [1, 2, 3, 4];
const somme = nombres.reduce((total, n) => total + n, 0); // 10
```

reverse() - inverser l'ordre du tableau

La méthode `.reverse()` inverse l'ordre des éléments d'un tableau. Elle modifie directement le tableau original.

```
const nombres = [1, 2, 3, 4];  
nombres.reverse(); // [4, 3, 2, 1]
```

Techniques

` (backticks) - pour des expressions intelligentes

Les backticks (`) servent à créer des chaînes de caractères dynamiques appelées template literals. Elles permettent d'insérer facilement des variables ou des expressions avec `${...}`.

```
const nom = 'Alex';  
const age = 25;  
console.log(`Bonjour ${nom}, tu as ${age} ans.`);  
// Bonjour Alex, tu as 25 ans.
```

new Set() - pour supprimer les doublons

Set est une structure qui ne garde que les valeurs uniques. En le combinant avec l'opérateur `...` (spread), on peut facilement supprimer les doublons d'un tableau.

```
const nombres = [1, 2, 2, 3, 4, 4, 5];  
const uniques = [...new Set(nombres)]; // [1, 2, 3, 4, 5]
```

Fonctions

Déclaration de fonction

Une fonction permet de regrouper un ensemble d'instructions réutilisables. Elle peut recevoir des paramètres et retourner une valeur avec `return`.

Standard

```
function doStuff(a, b, c) {  
  return a + b + c;  
}
```

Sous forme d'expression de fonction

Expression de fonction classique : utile quand on veut déclarer une fonction dans une variable.

```
const doStuff = function (a, b, c) {  
  return a + b + c;  
};
```

Sous forme d'expression de fonction anonyme Fonction fléchée (`=>`) : plus concise, conserve le contexte (`this`), souvent utilisée dans les callbacks.

```
const doStuff = (a, b, c) => {  
  return a + b + c;  
};
```

Sous forme raccourcie Version raccourcie : parfaite pour une fonction simple avec une seule instruction.

S'il n'y a qu'un seul argument et que son corps n'a qu'une seule expression, on peut omettre le `return` et le corps de la fonction :

```
const doStuff = (a) => `Salut ${a} !`;
```

Fonctions immédiatement invoquées (IIFE)

IIFE = Immediately Invoked Function Expressions.

Ces fonctions sont définies et **exécutées immédiatement**. Elles sont souvent utilisées pour créer un **contexte isolé** ou encapsuler du code sans polluer l'espace global.

```
(function(){ ... })()
```

ou

```
(( ) => { ... })()
```

Conclusion

Ce module de programmation fonctionnelle a été un véritable défi. D'être capable de reprendre ce que nous faisons jusque-là en programmation en utilisant une logique différente a vraiment été difficile pour moi. Nous avons appris à raisonner autrement, en privilégiant les fonctions pures, les immutabilités et les méthodes comme `map()`, `filter()`, ou `reduce()` plutôt que les boucles. Cela demande de bien comprendre la logique des transformations de données et la manière de chaîner les opérations pour obtenir le résultat voulu. Même si cela a été complexe au début, ce module m'a permis d'acquérir une nouvelle façon de penser le code, plus structurée, plus claire et souvent plus concise.